

BÀI 6: KIỂM THỬ DÒNG ĐIỀU KHIỂN

Mục tiêu:

- **Về kiến thức:** cung cấp cho sinh viên kiến thức tổng quan về kiểm thử hộp trắng, kiến thức về kỹ thuật kiểm thử dòng điều khiển: khái niệm, các thành phần của đồ thị dòng điều khiển, các độ đo kiểm thử dòng điều khiển
- **Về kỹ năng:** giúp sinh viên vẽ được đồ thị dòng điều khiển cho các đoạn vị mã nguồn cụ thể, liệt kê các đường thi hành tuyến tính độc lập thoả mãn độ phức tạp của đồ thị đã vẽ

6.1 Tổng quan về kiểm thử hộp trắng

Kiểm thử hộp trắng sử dụng các chiến lược cụ thể và sử dụng mã nguồn của chương trình/đơn vị phần mềm cần kiểm thử nhằm kiểm tra xem chương trình/đơn vị phần mềm có thực hiện đúng so với thiết kế và đặc tả hay không. Trong khi các phương pháp kiểm thử hộp đen hay kiểm thử hàm/chức năng chỉ cho phép phát hiện các lỗi/khiếm khuyết có thể quan sát được, kiểm thử hộp trắng cho phép phát hiện các lỗi/khiếm khuyết tiềm ẩn bên trong chương trình/đơn vị phần mềm. Các lỗi này thường khó phát hiện bởi các phương pháp kiểm thử hộp đen. Kiểm thử hộp đen và kiểm thử hộp trắng không thể thay thế cho nhau mà chúng cần được sử dụng kết hợp với nhau trong một quy trình kiểm thử thống nhất nhằm đảm bảo chất lượng phần mềm. Tuy nhiên, để áp dụng các phương pháp kiểm thử hộp trắng, người kiểm thử không chỉ cần hiểu rõ giải thuật mà còn cần có các kỹ năng và kiến thức tốt về ngôn ngữ lập trình được dùng để phát triển phần mềm, nhằm hiểu rõ mã nguồn của chương trình/đơn vị phần mềm cần kiểm thử. Do vậy, việc áp dụng các phương pháp kiểm thử hộp trắng thường tốn thời gian và công sức nhất là khi chương trình/đơn vị phần mềm có kích thước lớn. Vì lý do này, các phương pháp kiểm thử hộp trắng chủ yếu được sử dụng cho kiểm thử đơn vị.

6.2 Kiểm thử dựa trên dòng điều khiển

6.2.1 Đồ thị dòng điều khiển

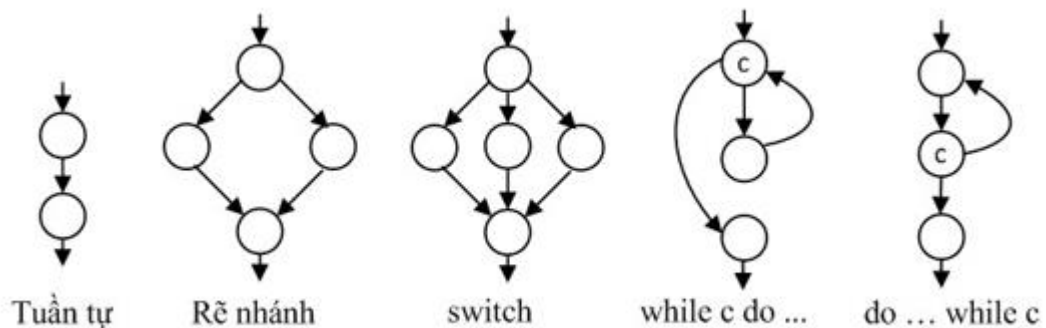
Phương pháp kiểm thử dòng điều khiển dựa trên khái niệm đồ thị dòng điều khiển. Đồ thị này được xây dựng từ mã nguồn của chương trình/đơn vị chương trình. Đồ thị dòng điều khiển là một đồ thị có hướng gồm các đỉnh tương ứng với các câu lệnh/nhóm

câu lệnh và các cạnh là các dòng điều khiển giữa các câu lệnh/nhóm câu lệnh. Nếu i và j là các đỉnh của đồ thị dòng điều khiển thì tồn tại một cạnh từ i đến j nếu lệnh tương ứng với j có thể được thực hiện ngay sau lệnh tương ứng với i .

Xây dựng một đồ thị dòng điều khiển từ một chương trình/đơn vị chương trình khá đơn giản. Hình 6.1 mô tả các thành phần cơ bản của đồ thị dòng điều khiển bao gồm điểm bắt đầu của đơn vị chương trình, khối xử lý chứa các câu lệnh khai báo hoặc tính toán, điểm quyết định ứng với các câu lệnh điều kiện trong các khối lệnh rẽ nhánh hoặc lặp, điểm nối ứng với các câu lệnh ngay sau các lệnh rẽ nhánh, và điểm kết thúc ứng với điểm kết thúc của đơn vị chương trình. Các cấu trúc điều khiển phổ biến của chương trình được mô tả trong Hình 6.2. Chúng ta sẽ sử dụng các thành phần cơ bản và các cấu trúc phổ biến này để dễ dàng xây dựng đồ thị dòng điều khiển cho mọi đơn vị chương trình viết bằng mọi ngôn ngữ lập trình.

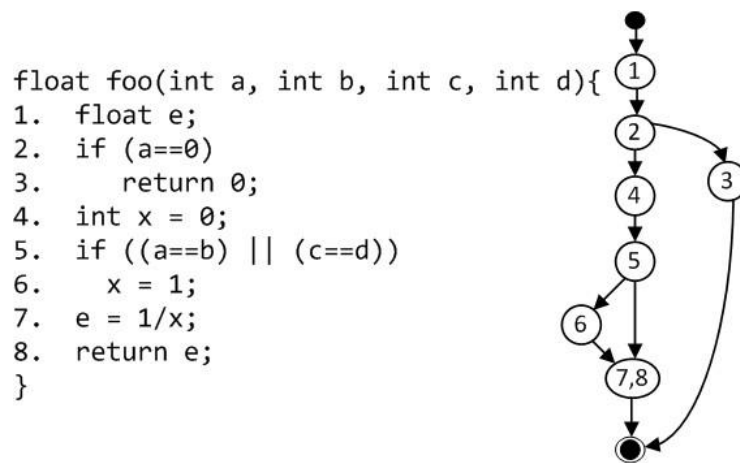


Hình 6.1: Các thành phần cơ bản của đồ thị chương trình.



Hình 6.2: Các cấu trúc điều khiển phổ biến của chương trình.

Chúng ta thử xem cách dựng đồ thị dòng điều khiển cho đơn vị chương trình có mã nguồn bằng ngôn ngữ C như Hình 6.3. Chúng ta đánh số các dòng lệnh của đơn vị chương trình và lấy số này làm đỉnh của đồ thị. Điểm xuất phát của đơn vị chương trình ứng với câu lệnh khai báo hàm foo. Đỉnh 1 ứng với câu lệnh khai báo biến e . Các đỉnh 2 và 3 ứng với câu lệnh `if`. Đỉnh 4 ứng với câu lệnh khai báo biến x trong khi các đỉnh 5 và 6 ứng với câu lệnh `if`. Đỉnh 7,8 đại diện cho hai câu lệnh 7 và 8. Trong trường hợp này, chúng ta không tách riêng thành hai đỉnh vì đây là hai câu lệnh tuần tự nên chúng ta ghép chúng thành một đỉnh nhằm tối thiểu số đỉnh của đồ thị dòng điều khiển.



Hình 6.3: Mã nguồn của hàm foo và đồ thị dòng điều khiển của nó.

6.2.2 Các độ đo kiểm thử

Kiểm thử hàm (kiểm thử hộp đen) có hạn chế là chúng ta không biết có thừa hay thiếu các ca kiểm thử hay không so với chương trình cài đặt và thiếu thừa ở mức độ nào. Độ đo kiểm thử là một công cụ giúp ta đo mức độ bao phủ chương trình của một tập ca kiểm thử cho trước. Mức độ bao phủ của một bộ kiểm thử (tập các ca kiểm thử) được đo bằng tỷ lệ các thành phần thực sự được kiểm thử so với tổng thể sau khi đã thực hiện các ca kiểm thử. Thành phần liên quan có thể là câu lệnh, điểm quyết định, điều kiện con, đường thi hành hay là sự kết hợp của chúng. Độ bao phủ càng lớn thì độ tin cậy của bộ kiểm thử càng cao. Độ đo này giúp chúng ta kiểm soát và quản lý quá trình kiểm thử tốt hơn. Mục tiêu của chúng ta là kiểm thử với số ca kiểm thử tối thiểu nhưng đạt được độ bao phủ tối đa. Có rất nhiều độ đo kiểm thử đang được sử dụng hiện nay, dưới đây là ba độ đo kiểm thử đang được sử dụng phổ biến nhất trong thực tế.

Độ đo kiểm thử cấp 1 (C1): mỗi câu lệnh được thực hiện ít nhất một lần sau khi chạy các ca kiểm thử (test cases). Ví dụ, với hàm foo có mã nguồn như trong Hình 4.3, ta chỉ cần hai ca kiểm thử như Bảng 6.1 là đạt 100% độ phủ cho độ đo C1 với EO (expected output) là giá trị đầu ra mong đợi và RO (real output) là giá trị đầu ra thực tế (giá trị này sẽ được điền khi thực hiện ca kiểm thử).

Bảng 6.1: Các ca kiểm thử cho độ đo C1 của hàm foo

ID	Inputs	EO	RO	Note
tc1	0, 1, 2, 3	0		
tc2	1, 1, 2, 3	1		

Độ đo kiểm thử cấp 2 (C2): các điểm quyết định trong đồ thị dòng điều khiển của đơn vị kiểm thử đều được thực hiện ít nhất một lần cả hai nhánh đúng và sai. Ví dụ, Bảng

6.2 mô tả các trường hợp cần kiểm thử để đạt được 100% độ phủ của độ đo C2 ứng với hàm foo được mô tả trong Hình 6.3.

Như vậy, với hai ca kiểm thử trong độ đo kiểm thử cấp 1 (tc1 và tc2), ta chỉ kiểm thử được $3/4 = 75\%$ ứng với độ đo kiểm thử cấp 2. Chúng ta cần một ca kiểm thử nữa ứng với trường hợp sai của điều kiện $(a == b) \parallel (c == d)$ nhằm đạt được 100% độ phủ của độ đo C2. Bảng 6.2 mô tả các ca kiểm thử cho mục đích này.

Bảng 6.2: Các ca kiểm thử cho độ đo C2 của hàm foo

ID	Inputs	EO	RO	Note
tc1	0, 1, 2, 3	0		
tc2	1, 1, 2, 3	1		
tc3	1, 2, 1, 2	Lỗi chia cho 0		

Độ đo kiểm thử cấp 3 (C3): Với các điều kiện phức tạp (chứa nhiều điều kiện con cơ bản), việc chỉ quan tâm đến giá trị đúng sai là không đủ để kiểm tra tính đúng đắn của chương trình ứng với điều kiện phức tạp này. Ví dụ, nếu một điều kiện phức tạp gồm hai điều kiện con cơ bản, chúng ta có bốn trường hợp cần kiểm thử chứ không phải hai trường hợp đúng sai như độ đo C2. Với các đơn vị chương trình có yêu cầu cao về tính đúng đắn, việc tuân thủ độ đo C3 là hết sức cần thiết. Điều kiện để đảm bảo độ đo này là các điều kiện con thuộc các điều kiện phức tạp tương ứng với các điểm quyết định trong đồ thị dòng điều khiển của đơn vị cần kiểm thử đều được thực hiện ít nhất một lần cả hai nhánh đúng và sai. Ví dụ, Bảng 6.3 mô tả các trường hợp cần kiểm thử để đạt được 100% độ phủ của độ đo C3 ứng với hàm foo được mô tả trong Hình 6.3.

Như vậy, với ba ca kiểm thử trong độ đo kiểm thử cấp 2 (tc1, tc2 và tc3), ta chỉ kiểm thử được $7/8 = 87,5\%$ ứng với độ đo kiểm thử cấp 3. Chúng ta cần một ca kiểm thử nữa ứng với trường hợp sai của điều kiện con cơ bản $(c == d)$ nhằm đạt được 100% độ phủ của độ đo C3. Bảng 6.4 mô tả các ca kiểm thử cho mục đích này.

Bảng 6.3: Các trường hợp cần kiểm thử của độ đo C3 với hàm foo

Điểm quyết định	Điều kiện tương ứng	Đúng	Sai
2	$a == 0$	tc1	tc2
5	$(a == b)$	tc2	tc3
5	$(c == d)$	?	tc2

Bảng 6.4: Các ca kiểm thử cho độ đo C3 của hàm foo

ID	Inputs	EO	RO	Note
tc1	0, 1, 2, 3	0		

tc2	1, 1, 2, 3	1		
tc3	1, 2, 1, 2	Lỗi chia cho 0		
tc4	1, 2, 1, 1	1		

Câu hỏi cuối bài:

Câu 1. Kiểm thử hộp trắng là gì?

Câu 2. So sánh kiểm thử hộp đen và kiểm thử hộp trắng?

Câu 3. Định nghĩa đồ thị dòng điều khiển?

Câu 4. Nêu các độ đo kiểm thử?

Câu 5. Các bài tập thực hành

Bài 1: Cho mã nguồn của hàm Xử lý phân loại như sau:

```
const char* classify_number(int x) {
    if (x < 10) return "Small";
    else if (x <= 100) return "Medium";
    else if (x > 100 && x <= 1000) return "Large";
    else return "Extra Large";
}
```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển đảm bảo độ đo C2, C3.
2. Tính độ phức tạp của mỗi đồ thị
3. Sinh testcase đảm bảo độ đo C2, C3

Bài 2: Cho mã nguồn của hàm Tính cước vận chuyển như sau:

```
float calculate_shipping_cost(float weight, int is_express) {
    if (weight <= 1.0) return 5.0;
    else if (weight <= 5.0) return 10.0;
    else {
        if (is_express) return weight * 3.0;
        else return weight * 2.0;
    }
}
```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển đảm bảo độ đo C1, C2

2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C1, C2

Bài 3: Cho mã nguồn của hàm Phân loại tam giác như sau:

```
const char* classify_triangle(int a, int b, int c) {  
    if (a + b <= c || a + c <= b || b + c <= a) return "Invalid";  
    if (a == b && b == c) return "Equilateral";  
    if (a == b || b == c || a == c) return "Isosceles";  
    return "Scalene";  
}
```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển đảm bảo độ đo C3
2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C3

Bài 4: Cho mã nguồn của hàm Kiểm tra số hoàn hảo như sau:

```
bool is_perfect_number(int n) {  
    if (n <= 1) return false;  
    int sum = 0;  
    for (int i = 1; i < n; i++) {  
        if (n % i == 0) sum += i;  
    }  
    return sum == n;  
}
```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển.
2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C2 và độ đo vòng lặp

Bài 5: Cho mã nguồn của hàm Xử lý nhập điểm như sau:

```
const char* validate_score(float score) {  
    if (score < 0 || score > 10) return "Invalid";  
    else if (score >= 9) return "Excellent";  
}
```

```

else if (score >= 7) return "Good";
else if (score >= 5) return "Average";
else return "Fail";
}

```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển đảm bảo độ đo C2 và C3
2. Tính độ phức tạp của mỗi đồ thị
3. Sinh testcase đảm bảo độ đo C2 và C3

Bài 6: Cho mã nguồn của hàm Tìm số lớn thứ hai như sau:

```

int find_second_largest(int arr[], int size) {
    int largest = -1, second_largest = -1;
    for (int i = 0; i < size; i++) {
        if (arr[i] > largest) {
            second_largest = largest;
            largest = arr[i];
        } else if (arr[i] > second_largest && arr[i] != largest) {
            second_largest = arr[i];
        }
    }
    return second_largest;
}

```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển đảm bảo độ đo C2 và C3
2. Tính độ phức tạp của mỗi đồ thị
3. Sinh testcase đảm bảo độ đo C2 và C3

Bài 7: Cho mã nguồn của hàm Kiểm tra chuỗi con như sau:

```

bool is_substring(const char* str, const char* substr) {
    int len_str = strlen(str);
    int len_substr = strlen(substr);
    for (int i = 0; i <= len_str - len_substr; i++) {

```

```

    int j;
    for (j = 0; j < len_substr; j++) {
        if (str[i + j] != substr[j]) break;
    }
    if (j == len_substr) return true;
}
return false;
}

```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển
2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C2 và độ đo vòng lặp bên trong

Bài 8: Cho mã nguồn của hàm Xác định số hoàn hảo trong một dãy như sau:

```

void find_perfect_numbers(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        int sum = 0;
        for (int j = 1; j < arr[i]; j++) {
            if (arr[i] % j == 0) sum += j;
        }
        if (sum == arr[i]) printf("%d is a perfect number\n", arr[i]);
    }
}

```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển
2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C2 và độ đo vòng lặp bên trong

Bài 9: Cho mã nguồn của hàm Sắp xếp mảng theo thứ tự tăng dần như sau:

```

void bubble_sort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {

```

```

        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}

```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển
2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C2 và độ đo vòng lặp bên trong

Bài 10: Cho mã nguồn của hàm Tính giai thừa với kiểm tra lỗi như sau:

```

int factorial(int n) {
    if (n < 0) return -1; // Lỗi
    if (n == 0) return 1;
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}

```

Yêu cầu:

1. Vẽ đồ thị dòng điều khiển
2. Tính độ phức tạp của đồ thị
3. Sinh testcase đảm bảo độ đo C2 và độ đo vòng lặp